Exp 6:-

Code:

```
        self.data = {}            # Key-value storage
class Bucket:
    def __init__(self, depth: int, capacity: int):
        self.depth = depth         # Local depth
        self.capacity = capacity    # Maximum items before split

    def is_full(self) -> bool:
        return len(self.data) >= self.capacity



class ExtendibleHashTable:
    def __init__(self, bucket_capacity: int = 2):
        self.global_depth = 1
        self.bucket_capacity = bucket_capacity

        # Start with 2^1 = 2 directory entries pointing to separate buckets
        b0 = Bucket(depth=1, capacity=bucket_capacity)
        b1 = Bucket(depth=1, capacity=bucket_capacity)
        self.directory = [b0, b1]

    def _hash(self, key) -> int:
        # Standard hash masked to positive integer
        return hash(key) & 0x7FFFFFFF

    def _get_bucket_index(self, key) -> int:
        # Mask using the lowest 'global_depth' bits
        mask = (1 << self.global_depth) - 1
```

```
    return self._hash(key) & mask

def search(self, key):
    """Returns the value associated with key, or None if not found."""
    idx = self._get_bucket_index(key)
    return self.directory[idx].data.get(key, None)

def insert(self, key, value):
    """Inserts or updates a key-value pair, handling splits and directory doubling."""
    idx = self._get_bucket_index(key)
    bucket = self.directory[idx]

    # Key already exists: update in place without increasing count
    if key in bucket.data:
        bucket.data[key] = value
        return

    # Fits in current bucket: store directly
    if not bucket.is_full():
        bucket.data[key] = value
        return

    # Bucket is full: split required
    self._split_bucket(idx)
    # Re-attempt insertion recursively into the updated directory structure
    self.insert(key, value)

def _split_bucket(self, index: int):
    target_bucket = self.directory[index]
```

```
        # Case 1: Local depth equals global depth -> Double the directory
        if target_bucket.depth == self.global_depth:
            self._double_directory()

        # Allocate new bucket and increment depth for both
        new_depth = target_bucket.depth + 1
        target_bucket.depth = new_depth
        new_bucket = Bucket(depth=new_depth, capacity=self.bucket_capacity)

        # Reassign directory pointers
        # Any directory index sharing the old prefix with bit (new_depth - 1) set to 1 moves to new_bucket
        split_bit = 1 << (new_depth - 1)
        for i in range(len(self.directory)):
            if self.directory[i] is target_bucket:
                if (i & split_bit) != 0:
                    self.directory[i] = new_bucket

        # Redistribute existing keys between target_bucket and new_bucket
        old_items = list(target_bucket.data.items())
        target_bucket.data.clear()

        for k, v in old_items:
            mask = (1 << new_depth) - 1
            if (self._hash(k) & mask) & split_bit:
                new_bucket.data[k] = v
            else:
                target_bucket.data[k] = v
```

```
    def _double_directory(self):
        """Doubles directory size and updates global depth."""
        self.directory += list(self.directory)
        self.global_depth += 1

    def delete(self, key) -> bool:
        """Removes a key. Returns True if removed, False otherwise."""
        idx = self._get_bucket_index(key)
        bucket = self.directory[idx]
        if key in bucket.data:
            del bucket.data[key]
            return True
        return False

    def display(self):
        """Prints directory pointers and bucket states."""
        print(f"\n--- Directory (Global Depth: {self.global_depth}) ---")
        seen_buckets = set()
        for i, bucket in enumerate(self.directory):
            bin_idx = bin(i)[2:].zfill(self.global_depth)
            bucket_id = id(bucket)
            print(f"Dir[{bin_idx}] -> Bucket @ {hex(bucket_id)[-5:]} (Local Depth:
{bucket.depth})")

        print("\n--- Unique Buckets ---")
        for bucket in self.directory:
            if bucket not in seen_buckets:
                seen_buckets.add(bucket)
                print(f"Bucket @ {hex(id(bucket))[-5:]} [Depth: {bucket.depth}]: {bucket.data}")
```

```
# --- Verification ---
if __name__ == "__main__":
    ht = ExtendibleHashTable(bucket_capacity=2)

    # Insert items to demonstrate bucket splits and directory expansion
    data_to_insert = [
        ("apple", 10),
        ("banana", 20),
        ("cherry", 30),
        ("date", 40),
        ("elderberry", 50),
        ("fig", 60),
    ]

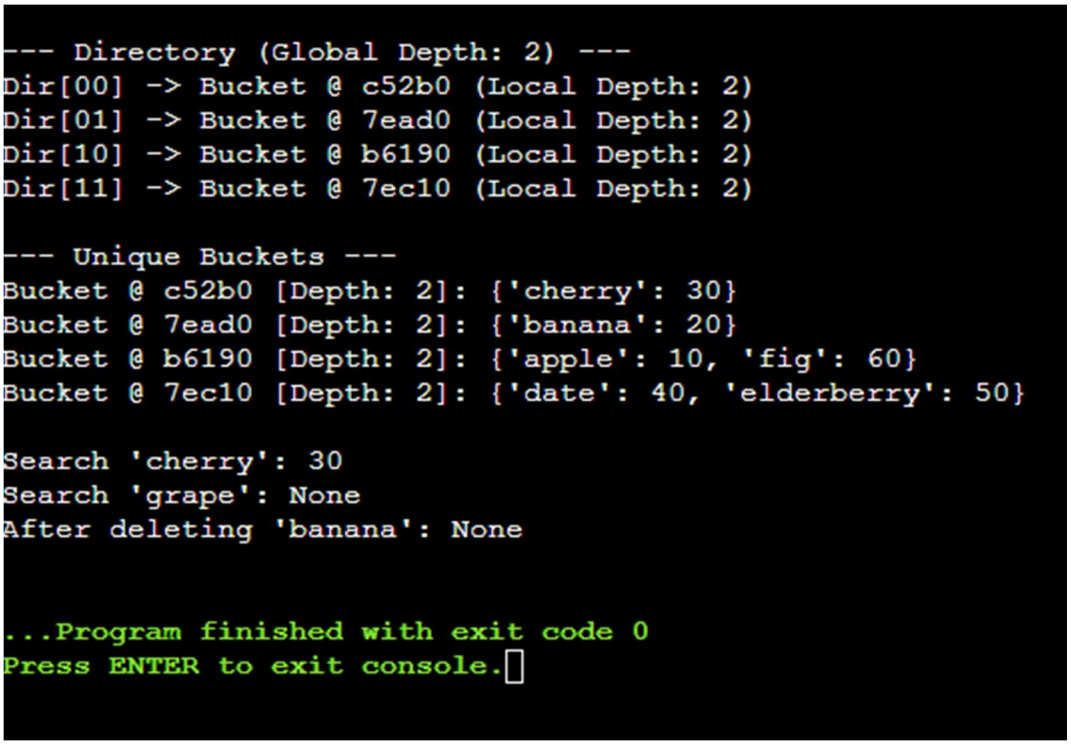
    for k, v in data_to_insert:
        ht.insert(k, v)

    ht.display()

    # Search tests
    print("\nSearch 'cherry':", ht.search("cherry"))
    print("Search 'grape':", ht.search("grape"))

    # Deletion
    ht.delete("banana")
    print("After deleting 'banana':", ht.search("banana"))
```

output


--- Directory (Global Depth: 2) ---
Dir[00] -> Bucket @ c52b0 (Local Depth: 2)
Dir[01] -> Bucket @ 7ead0 (Local Depth: 2)
Dir[10] -> Bucket @ b6190 (Local Depth: 2)
Dir[11] -> Bucket @ 7ec10 (Local Depth: 2)
--- Unique Buckets ---
Bucket @ c52b0 [Depth: 2]: {'cherry': 30}
Bucket @ 7ead0 [Depth: 2]: {'banana': 20}
Bucket @ b6190 [Depth: 2]: {'apple': 10, 'fig': 60}
Bucket @ 7ec10 [Depth: 2]: {'date': 40, 'elderberry': 50}
Search 'cherry': 30
Search 'grape': None
After deleting 'banana': None
...Program finished with exit code 0
Press ENTER to exit console.